SNAKE GAME

Complete Source Code Guide

Line-by-Line Explanation of Every File

By Kethmika Edirisinghe

Language	C (C99)
Platform	Windows 11 CMD
Files	common.h, logic, input, render, main

How All Files Connect

The game is split into 5 pairs of files (header + source). Each file has one clear job. They talk to each other only through the functions declared in the header files.

Code	Explanation
`common.h`	Shared types used by every file. Vec2, MAX sizes.
`logic.h/.c`	Game brain. Stores all state, moves the snake, handles eating.
`input.h/.c`	Reads the keyboard. Feeds direction into logic.
`render.h/.c`	Draws the grid to CMD. Knows about logic state but never changes it.
`main.c`	The game loop. Calls the other 3 in order every frame.

Think of it like a kitchen: logic.c is the chef (makes decisions), input.c is the waiter (takes orders from you), render.c is the serving station (shows the result), and main.c is the timer that keeps everything in sync.

The Game Loop — What Happens Every Frame

Every tick of the game (every 120ms), main.c does these 4 things in order:

Code	Explanation
`poll_input()`	Drain keyboard buffer. Queue latest direction pressed.
`update_game()`	Move snake head, check wall/self collision, eat fruit if on it.
`render()`	Build the character grid, overwrite the previous frame on screen.
`Sleep(120)`	Wait 120ms before next tick. Controls game speed.

File 1: common.h — Shared Types

A header file (.h) is just a list of declarations. It tells other files what types and functions exist WITHOUT containing the actual code. Every .c file includes the headers it needs.

```
#ifndef COMMON_H
#define COMMON_H
#include <stdbool.h>

#define MAX_SNAKE_LEN 512
#define MAX_GRID_W     80
#define MAX_GRID_H     40

typedef struct { int x, y; } Vec2;
#endif
```

Line by Line

Code	Explanation
`#ifndef COMMON_H`	Include guard start. If this file was already included, skip everything inside. Prevents the same code being defined twice.
`#define COMMON_H`	Marks this file as 'already included' so the ifndef above fires next time.
`#include <stdbool.h>`	Brings in bool, true, false. These are standard C99 types needed throughout the project.
`#define MAX_SNAKE_LEN 512`	Constant. The snake body array is this size. Snake can never grow longer than 512 segments. Change this number to allow longer snakes.
`#define MAX_GRID_W  80`	Constant. Maximum grid width the render buffer can hold (columns).
`#define MAX_GRID_H  40`	Constant. Maximum grid height the render buffer can hold (rows).
`typedef struct {int x,y;} Vec2;`	Defines a new type called Vec2 — a pair of integers. Used for positions (where is the snake?) and directions (which way is it going?). x = column, y = row.
`#endif`	End of the include guard block.

Vec2 is used everywhere. A direction is also a Vec2: right = {1,0}, left = {-1,0}, up = {0,-1}, down = {0,1}. Moving means adding the direction Vec2 to the position Vec2.

File 2: logic.h — Game State Declaration

This header declares the Game struct (all game data in one place) and the function signatures for logic.c.

```
#ifndef LOGIC_H
#define LOGIC_H
#include "common.h"

typedef struct {
    Vec2 body[MAX_SNAKE_LEN];
    int  length;
    Vec2 dir;
    Vec2 next_dir;
    Vec2 fruit;
    int  score;
    int  width, height;
} Game;

void init_game   (Game *g, int w, int h);
void update_game (Game *g, bool *running);
void spawn_fruit (Game *g);
bool is_on_snake (const Game *g, Vec2 pos);
#endif
```

The Game Struct — Line by Line

A struct is a container that groups related variables together. Instead of having 10 separate variables floating around, they all live inside one Game variable. Pass &g to any function and it can read/write any piece of state.

Code	Explanation
`Vec2 body[MAX_SNAKE_LEN]`	Array of 512 Vec2 positions. body[0] is always the HEAD. body[1] is the segment behind the head, body[2] behind that, etc. Only 'length' entries are active.
`int length`	How many body segments are currently alive. Starts at 3. Goes up by 1 each time fruit is eaten. Maximum is MAX_SNAKE_LEN.
`Vec2 dir`	The direction the snake IS moving this tick. Gets committed from next_dir at the start of update_game().
`Vec2 next_dir`	The direction queued by the last key press. Kept separate from dir so if you press two keys in one tick, you don't skip through walls.
`Vec2 fruit`	The x,y position of the current fruit on the grid.
`int score`	Number of fruits eaten so far. Shown on the status bar.
`int width, height`	Grid dimensions including the border walls. Grid cells the snake can actually move in are (1 to width-2) and (1 to height-2).

Function Declarations

Code	Explanation
`void init_game(Game *g, int w, int h)`	Sets up the game from scratch. Places snake in the centre, seeds random number generator, spawns first fruit.
`void update_game(Game *g, bool *running)`	Advances one tick. Moves the snake, checks collisions, handles eating. Sets *running=false to end the game.
`void spawn_fruit(Game *g)`	Picks a random empty cell and places the fruit there.
`bool is_on_snake(const Game *g, Vec2 pos)`	Returns true if a given position overlaps any part of the snake body. Used by spawn_fruit to avoid placing fruit inside the snake.

File 3: logic.c — Game Logic

This is the brain of the game. It implements the 4 functions declared in logic.h. No drawing happens here — only state changes.

is_on_snake()

```
bool is_on_snake(const Game *g, Vec2 pos) {
    for (int i = 0; i < g->length; i++)
        if (g->body[i].x == pos.x && g->body[i].y == pos.y)
            return true;
    return false;
}
```

Code	Explanation
`const Game *g`	Pointer to the game. 'const' means this function promises not to change anything — read-only.
`for (int i = 0; i < g->length; i++)`	Loop through every active body segment. Stop at length, not MAX_SNAKE_LEN, so we don't check empty slots.
`if (body[i].x == pos.x && body[i].y == pos.y)`	If both x AND y match, this body segment is at the given position.
`return true / return false`	True if any segment matched, false if we checked all segments with no match.

spawn_fruit()

```
void spawn_fruit(Game *g) {
    Vec2 pos;
    int tries = 0;
    do {
        pos.x = 1 + rand() % (g->width  - 2);
        pos.y = 1 + rand() % (g->height - 2);
    } while (is_on_snake(g, pos) && ++tries < 2000);
    g->fruit = pos;
}
```

Code	Explanation
`int tries = 0`	Safety counter. If the snake fills almost the entire grid, avoid an infinite loop.
`do { ... } while (...)`	Do-while runs the body FIRST, then checks the condition. We always pick at least one position before checking if it's valid.
`1 + rand() % (g->width - 2)`	rand() gives a random number. % (width-2) limits it to 0..width-3. Adding 1 shifts it to 1..width-2. This keeps fruit away from the border walls (which are at x=0 and x=width-1).

`while (is_on_snake(g, pos) && ++tries < 2000)`	If the chosen cell is inside the snake, pick again. Stop after 2000 tries to prevent infinite loop on nearly-full board.
`g->fruit = pos`	Save the chosen position into the game state.

init_game()

```
void init_game(Game *g, int w, int h) {
    srand((unsigned)time(NULL));
    g->width  = w;
    g->height = h;
    g->length   = 3;
    g->body[0]  = (Vec2){ w/2,     h/2 };
    g->body[1]  = (Vec2){ w/2 - 1, h/2 };
    g->body[2]  = (Vec2){ w/2 - 2, h/2 };
    g->dir      = (Vec2){ 1, 0 };
    g->next_dir = (Vec2){ 1, 0 };
    g->score    = 0;
    spawn_fruit(g);
}
```

Code	Explanation
`srand((unsigned)time(NULL))`	Seeds the random number generator with the current time. Without this, rand() gives the same sequence every run — the fruit would always appear in the same spots.
`g->width = w; g->height = h`	Store the grid dimensions. All other logic uses these so the grid size can be changed in main.c without touching logic.c.
`g->length = 3`	Start with a 3-segment snake instead of 1. Makes the reverse-direction kill immediately visible on game start.
`body[0] = {w/2, h/2}`	Head starts in the exact centre of the grid.
`body[1] = {w/2-1, h/2}`	First body segment — one column to the LEFT of the head (the snake was coming from the left).
`body[2] = {w/2-2, h/2}`	Second body segment — two columns left of the head. The snake looks like: oo@ moving right.
`dir = next_dir = {1, 0}`	Start moving RIGHT. x=1 means move one column right per tick. Both dir and next_dir set so there is no 'waiting for first input' phase.
`g->score = 0`	Reset score to zero.
`spawn_fruit(g)`	Place the first fruit before the game loop starts.

update_game() — The Core Movement Logic

```
void update_game(Game *g, bool *running) {
    g->dir = g->next_dir;
```

```
    Vec2 nh = { g->body[0].x + g->dir.x,
                g->body[0].y + g->dir.y };

    if (nh.x <= 0 || nh.x >= g->width  - 1 ||
        nh.y <= 0 || nh.y >= g->height - 1) {
        *running = false; return;
    }

    for (int i = 0; i < g->length - 1; i++) {
        if (body[i].x == nh.x && body[i].y == nh.y) {
            *running = false; return;
        }
    }

    bool ate = (nh.x == g->fruit.x && nh.y == g->fruit.y);
    if (ate && g->length < MAX_SNAKE_LEN) g->length++;

    for (int i = g->length - 1; i > 0; i--)
        g->body[i] = g->body[i - 1];
    g->body[0] = nh;

    if (ate) { g->score++; spawn_fruit(g); }
}
```

Code	Explanation
`g->dir = g->next_dir`	Commit the queued direction. We separate 'queued' from 'current' so a key press during render/sleep doesn't take effect mid-frame.
`Vec2 nh = {head.x + dir.x, head.y + dir.y}`	Calculate where the head WILL move. We don't move it yet — we check collisions first before committing.
`if (nh.x <= 0 || nh.x >= width-1 ...)`	Border wall check. The walls ARE at x=0 and x=width-1 (they are drawn as '#'). Using <= 0 and >= width-1 means hitting the wall cell itself is death.
`for i = 0 to length-2`	Self-collision check. Loop through body segments. Stop at length-1 (exclude the tail segment deliberately — explained below).
`Exclude the tail segment`	The tail is about to move away this same tick. If the head enters where the tail was, it's valid. Including the tail causes false deaths when turning sharply.
`bool ate = (nh == fruit)`	Check if the new head position lands exactly on the fruit. Checked BEFORE the body shift.
`if (ate) length++`	Grow by 1. The body shift loop runs up to the new length, so one extra copy survives at the end — the tail stays in place this tick.
`for i = length-1 down to 1: body[i] = body[i-1]`	The conveyor belt shift. Each segment copies the position of the segment in front of it. Runs from the TAIL backward to avoid overwriting.
`body[0] = nh`	Place the head at the new position. This is the last step — all collision checks are already done.
`if (ate) { score++; spawn_fruit() }`	After moving, if we ate: increment score and place a new fruit.

Why shift from tail to head? If you went head to tail you'd copy body[0] into body[1], then body[1] (now a copy of body[0]) into body[2], etc. — every segment ends up at the head position. Going backward ensures each segment gets a genuine copy of its predecessor before that predecessor changes.

File 4: input.h and input.c — Keyboard Handling

input.c reads the Windows keyboard buffer and converts key presses into direction changes. It also guards against illegal moves.

input.h — The Declaration

```
void input_init    (void);
void input_shutdown(void);
void poll_input(const Vec2 *current_dir, Vec2 *next_dir, bool *running);
```

Code	Explanation
`input_init / input_shutdown`	Setup and teardown. Currently empty — included so you can add terminal mode changes later without touching other files.
`const Vec2 *current_dir`	Read-only pointer to the direction the snake is CURRENTLY moving. Used to block 180-degree reversals.
`Vec2 *next_dir`	Pointer to the QUEUED direction. poll_input writes here. update_game reads from here next tick.
`bool *running`	Pointer to the running flag. Set to false if Q is pressed.

input.c — Full Implementation

```
void poll_input(const Vec2 *cur, Vec2 *next, bool *running) {
    while (_kbhit()) {
        int  c  = _getch();
        Vec2 nd = *next;

        switch (c) {
            case 'w': case 'W': nd = (Vec2){ 0, -1}; break;
            case 's': case 'S': nd = (Vec2){ 0,  1}; break;
            case 'a': case 'A': nd = (Vec2){-1,  0}; break;
            case 'd': case 'D': nd = (Vec2){ 1,  0}; break;
            case 'q': case 'Q': *running = false; return;
            default : continue;
        }

        if (nd.x == -cur->x && nd.y == -cur->y)
            continue;

        *next = nd;
    }
}
```

Code	Explanation

`while (_kbhit())`	_kbhit() returns non-zero if there is at least one key in the buffer. The while loop keeps reading until the buffer is empty. THIS was the big fix — old code used if() and only read one key per frame, causing input lag.
`int c = _getch()`	Read one key from the buffer without waiting. Returns the ASCII value of the key. Does not echo to the screen.
`Vec2 nd = *next`	Start with whatever direction was already queued. If this key is invalid (arrow key, etc.) we keep the existing queued direction unchanged.
`case 'w': nd = {0, -1}`	W = move UP. In screen coordinates, UP means decreasing y (row 0 is the top of the screen).
`case 's': nd = {0, +1}`	S = move DOWN. Increasing y moves toward the bottom of the screen.
`case 'a': nd = {-1, 0}`	A = move LEFT. Decreasing x moves toward the left edge.
`case 'd': nd = {+1, 0}`	D = move RIGHT. Increasing x moves toward the right edge.
`case 'q': *running=false; return`	Q quits immediately. We return (not break) to exit the entire function right away — no more keys are processed.
`default: continue`	Any other key (arrow keys, function keys, etc.) — skip it and check the next buffered key.
`if (nd.x == -cur->x && nd.y == -cur->y) continue`	Reversal guard. If the new direction is the exact opposite of current direction, discard it. Example: moving right (1,0), pressing A gives (-1,0). -cur->x = -1, nd.x = -1, they match → skip. Without this the snake head moves into its own neck and dies instantly.
`*next = nd`	Only if the direction passed all checks: store it as the new queued direction. This overwrites whatever was queued before, so the LAST valid key in the buffer wins.

Why does keeping the LAST key win matter? If you press W then D very quickly within one 120ms frame, both go into the buffer. The old code read only W and applied it. The new code reads W, stores it, reads D, stores it — so D wins. This matches what players expect: the most recent input takes effect.

File 5: render.h and render.c — Drawing to CMD

render.c is responsible for turning the game state into characters on the screen. It never changes game state — it only reads from Game and writes to stdout.

render.h — The Declaration

```
void render_init     (void);
void render_shutdown(void);
void render          (const Game *g);
```

Code	Explanation
`render_init()`	Called once at startup. Gets the console handle, hides the blinking cursor. Must be called before the first render().
`render_shutdown()`	Called once when the game ends. Restores the visible cursor so CMD looks normal again after the game exits.
`render(const Game *g)`	Called every frame. Reads game state, builds the screen, outputs it. const means it will NEVER change the game state.

render_init() and render_shutdown()

```
static HANDLE hOut;

void render_init(void) {
    hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO ci = {1, FALSE};
    SetConsoleCursorInfo(hOut, &ci);
}

void render_shutdown(void) {
    CONSOLE_CURSOR_INFO ci = {1, TRUE};
    SetConsoleCursorInfo(hOut, &ci);
}
```

Code	Explanation
`static HANDLE hOut`	'static' means this variable exists only in render.c. Other files cannot see it. HANDLE is a Windows type — a reference to a system resource.
`GetStdHandle(STD_OUTPUT_HANDLE)`	Ask Windows for a handle to the console output. We need this to call other Win32 console functions.
`CONSOLE_CURSOR_INFO ci = {1, FALSE}`	A struct with two fields: cursor size (1 = smallest) and visible (FALSE = hidden). Hiding it stops the cursor from flashing across the screen as we redraw.
`SetConsoleCursorInfo(hOut, &ci)`	Apply the cursor settings to our console window.

`render_shutdown: visible = TRUE`	Restore the cursor when the game ends. Without this, the CMD prompt after the game would have no cursor.

cursor_home() — The Flicker Fix

```
static void cursor_home(void) {
    COORD c = {0, 0};
    SetConsoleCursorPosition(hOut, c);
}
```

Code	Explanation
`COORD c = {0, 0}`	COORD is a Windows struct for a console position. {0,0} means column 0, row 0 — the very top-left of the terminal window.
`SetConsoleCursorPosition(hOut, c)`	Move the cursor to top-left WITHOUT clearing the screen. The next fwrite call overwrites the old content character by character. The screen is never blank — no flicker.

This is the most important performance fix. system("cls") (the old code) spawned a new CMD process, cleared the screen to black, then returned — causing the screen to flash blank each frame. cursor_home() is a single Win32 call, takes microseconds, and the screen never goes dark.

render() — Building and Outputting the Frame

```
void render(const Game *g) {
    char grid[MAX_GRID_H][MAX_GRID_W];
    memset(grid, ' ', sizeof(grid));

    // Top and bottom borders
    for (int x = 0; x < g->width; x++) {
        grid[0][x]            = '#';
        grid[g->height-1][x] = '#';
    }
    // Left and right borders
    for (int y = 0; y < g->height; y++) {
        grid[y][0]           = '#';
        grid[y][g->width-1] = '#';
    }

    // Fruit
    grid[g->fruit.y][g->fruit.x] = '$';

    // Snake body then head (head overwrites body on overlap)
    for (int i = g->length-1; i >= 0; i--) {
        Vec2 p = g->body[i];
        grid[p.y][p.x] = (i == 0) ? '@' : 'o';
    }

    // Build output buffer
    char out[MAX_GRID_H * (MAX_GRID_W+2) + 512];
    int  n = 0;
```

```
    for (int y = 0; y < g->height; y++) {
        for (int x = 0; x < g->width; x++)
            out[n++] = grid[y][x];
        out[n++] = '\n';
    }
    n += sprintf(out+n, "| Score: %d ...", g->score, ...);

    cursor_home();
    fwrite(out, 1, n, stdout);
    fflush(stdout);
}
```

Code	Explanation
`char grid[MAX_GRID_H][MAX_GRID_W]`	A 2D array of characters, one per grid cell. Allocated on the stack each frame — fast, no malloc needed.
`memset(grid, ' ', sizeof(grid))`	Fill the entire grid with spaces. This is the 'blank canvas' before we draw anything on top.
`Border loops`	Draw '#' on row 0, row height-1 (top and bottom), and column 0, column width-1 (left and right). These 4 edges form the visible box.
`grid[fruit.y][fruit.x] = '$'`	Place the fruit character at the fruit's position. Notice grid is indexed [row][column] = [y][x]. This is the standard convention for 2D arrays.
`for i = length-1 down to 0`	Draw snake from tail to head. Tail segments get 'o', head gets '@'. Going tail-to-head means if the head and a body segment ever occupy the same cell (shouldn't happen normally), the head '@' character wins because it's drawn last.
`char out[...]; int n = 0`	Output buffer. We assemble the entire frame as a string in memory before writing anything to the screen.
`out[n++] = grid[y][x]`	Copy each character from the grid into the output buffer. n tracks how many bytes we've written.
`out[n++] = '\n'`	Add a newline at the end of each row so rows appear on separate lines in CMD.
`sprintf(out+n, ...)`	Append the status bar (score, length, controls) to the end of the buffer. out+n points to the next empty byte.
`cursor_home()`	Jump cursor to top-left. The old frame is still visible behind the cursor.
`fwrite(out, 1, n, stdout)`	Write the entire frame — ALL bytes — in one single call. This is much faster than hundreds of putchar() calls and prevents partial-frame rendering.
`fflush(stdout)`	Force stdout to actually send its internal buffer to the console. Without this, some output might be held back and appear on the wrong frame.

File 6: main.c — The Game Loop

main.c is the conductor. It owns the game loop and calls the other modules in the correct order. It contains almost no logic of its own.

```
#include <stdio.h>
#include <stdbool.h>
#include <windows.h>
#include "common.h"
#include "logic.h"
#include "render.h"
#include "input.h"

int main(void) {
    Game g;
    bool running = true;

    render_init();
    input_init();
    init_game(&g, 40, 22);
    render(&g);

    while (running) {
        poll_input(&g.dir, &g.next_dir, &running);
        update_game(&g, &running);
        render(&g);
        Sleep(120);
    }

    render_shutdown();
    input_shutdown();
    printf("\n\n  *** GAME OVER ***  Score: %d\n\n", g.score);
    return 0;
}
```

Code	Explanation
`#include <stdio.h>`	Standard C I/O — provides printf, fwrite, fflush, etc.
`#include <stdbool.h>`	Provides the bool type and true/false values.
`#include <windows.h>`	Windows API — provides Sleep(), HANDLE, COORD, console functions. Only needed in main and render.
`#include "common.h" etc.`	Our own headers. Double quotes instead of angle brackets means the compiler looks in the same folder first.
`Game g;`	The one and only Game variable. All game state lives here. Passed by pointer (&g) to every function that needs it.
`bool running = true;`	The loop control flag. While true the game continues. poll_input sets it false on Q, update_game sets it false on collision.

`render_init()`	Must be called BEFORE the first render(). Gets the console handle and hides the cursor.
`input_init()`	Called for consistency — currently empty but part of the interface contract.
`init_game(&g, 40, 22)`	Set up the game: 40 columns wide, 22 rows tall. To make the grid bigger or smaller, change these two numbers only.
`render(&g)`	Initial draw BEFORE entering the loop. Without this, the screen is blank for the first 120ms.
`poll_input(&g.dir, &g.next_dir, &running)`	Pass pointers to dir, next_dir, and running. poll_input can modify next_dir and running through these pointers.
`update_game(&g, &running)`	Advance game state. May set running=false if the snake dies.
`render(&g)`	Draw the updated state. Called AFTER update so the player always sees the result of their last move.
`Sleep(120)`	Pause 120 milliseconds. Controls game speed. Increase for slower, decrease for faster. See the speed guide in the project notes.
`render_shutdown()`	Restore cursor after game ends. Called even on game-over so CMD is left in clean state.
`printf("GAME OVER...")`	Print the final message below the game grid (not inside the grid area, which was overwritten each frame).
`return 0`	Convention: return 0 from main means the program exited successfully.

Quick Reference — Common Changes

Code	Explanation
`Change game speed`	main.c → Sleep(120). Higher = slower. Lower = faster. Formula: moves per second = 1000 / Sleep value.
`Change grid size`	main.c → init_game(&g, WIDTH, HEIGHT). Minimum 10x10 to have playable space inside borders.
`Change max snake length`	common.h → #define MAX_SNAKE_LEN 512. Increase if you want longer snakes.
`Change starting snake length`	logic.c → init_game → g->length = 3. Change the number and add/remove matching body[] entries.
`Change fruit character`	render.c → grid[fruit.y][fruit.x] = '$'. Change '$' to any character.
`Change snake characters`	render.c → '@' for head, 'o' for body. Change to any characters you like.
`Change border character`	render.c → '#' in the border loops. Change to any character.
`Add increasing speed`	logic.c → in update_game after eating, reduce a delay variable stored in Game struct and use it in Sleep() in main.c.

Compile and Run

```
cd snake
compile.bat
snake.exe
```

compile.bat runs: gcc -O2 -Wall -o snake.exe main.c logic.c input.c render.c -O2 = optimise the code. -Wall = show all warnings. All 4 .c files must be listed.